Modular Static Scheduling of Synchronous Data-flow Networks

An efficient symbolic representation *

Marc Pouzet
LRI, Univ. Paris-Sud 11
Orsay
Marc.Pouzet@lri.fr

Pascal Raymond
VERIMAG-CNRS
Grenoble
Pascal.Raymond@imag.fr

ABSTRACT

This paper addresses the question of producing modular sequential imperative code from synchronous data-flow networks. Precisely, given a system with several input and output flows, how to decompose it into a minimal number of sub-systems executed atomically and statically scheduled without restricting possible feedback loops between input and output?

Though this question has been identified by Raymond in the early years of LUSTRE, it has almost been left aside until the recent work of Lublinerman, Szegedy and Tripakis. The problem is proven to be intractable, in the sense that it belongs to the family of optimization problems where the corresponding decision problem — there exists a solution with size c — is NP-complete. Then, the authors derive an iterative algorithm looking for solutions for $c = 1, 2, ...$ where each step is encoded as a SAT problem.

Despite the apparent intractability of the problem, our experience is that real programs do not exhibit such a complexity. Based on earlier work by Raymond, this paper presents a new symbolic encoding of the problem in terms of input/output relations. This encoding *simplifies* the problem, in the sense that it rejects solutions, while keeping all the optimal ones. It allows, in polynomial time, (1) to identify nodes for which several schedules are feasible and thus are possible sources of combinational explosion; (2) to obtain solutions which in some cases are already optimal; (3) otherwise, to get a non trivial lower bound for c to start an iterative combinational search.

The solution applies to a large class of block-diagram formalisms based on atomic computations and a *delay* operator, ranging from synchronous languages such as LUSTRE or SCADE to modeling tools such as SIMULINK.

*This work was partially supported by the INRIA research project Synchronics.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
EMSOFT'09, October 12–16, 2009, Grenoble, France.
Copyright 2009 ACM 978-1-60558-627-4/09/10 ...$10.00.

Categories and Subject Descriptors

C.3 [**Special-Purpose and Application-Based Systems**]: Real-time and embedded systems; D.3.2 [**Language Classifications**]: Data-flow languages; D.3.4 [**Processors**]: Code generation, Compilers

General Terms

Algorithms, Languages, Theory

Keywords

Real-time systems, Synchronous languages, Block-diagrams, Compilation, Semantics, NP-completeness, Partial orders, Preorders

1. INTRODUCTION

The synchronous block-diagram or *data-flow* formalism is now preeminent in a variety of design tools for embedded systems. Sequential code generation of synchronous block-diagrams have been considered in the early years of LUSTRE [7] and SIGNAL [1] and is provided by industrial tools such as SCADE [1] and RTBUILDER [2] for almost fifteen years. Though it has been considered more recently, modeling and simulation tools such as SIMULINK [3] and MODELICA [4] are now equipped with an automatic code generator.

We focus here on the problem of generating imperative, sequential code, implementing the functional behavior of a parallel data-flow network. We keep abstracted the (somehow orthogonal) problem of data management, that is, how values are actually passed from one node to another and even the interpretation of each operator. In particular, we address both data-flow networks with discrete-time (e.g., SCADE) or continuous-time (e.g., SIMULINK) semantics.

Whatever be the semantics of nodes in a network, there are basically two types of atomic nodes. *Combinatorial* nodes need to evaluate all their arguments in order to produce their output. On the contrary, *delay* nodes are able to produce their output before reading their inputs. They correspond to unitary registers in synchronous designs (the so-called `pre` operator of LUSTRE), initialized buffers in Kahn process networks [9, 10] or continuous integrators in SIMULINK.

[1] `http://www.esterel-technologies.com/scade/`
[2] `http://www.tni-software.com`
[3] `http://www.mathworks.com/product/simulink`
[4] `http://www.modelica.org`

Delays are essential to cut data-dependencies and to introduce well-founded feed-back loops.

Given a system described as a data-flow network of atomic operators, code generation aims at producing a static schedule satisfying data-dependencies. This static schedule is feasible when there is no combinatorial loop, i.e., every loop in the graph crosses a delay. Data-flow formalism, just like any advanced programming language, allow the user to abstract his own programs into reusable components, that is, to build functions. This raises the problem of defining what is exactly a reusable parallel program, and how to compile it once for all into sequential code. Modular, or separate compilation, produces one sequential code executed atomically for each function definition. Nonetheless, modular code generation is not always feasible even in the absence of combinatorial loops, as noticed by Gonthier [6]. If $copy(x, y) = (x + 1, y + 1)$ then the equation $(y, z) = copy(t, y)$ defines two perfectly valid streams y and z (since $y = t + 1$ and $z = y+1 = (t+1)+1$) but it cannot be statically scheduled if *copy* is compiled into one atomic step function. Indeed, it would make both y and z depend on t and y. This observation has conducted to two main compilation approaches. With the first one, or *white boxing*, functions are statically inlined before code generation. This is the solution taken in the academic LUSTRE compiler. The opposite solution or *black-boxing*, keeps maximal code sharing by compiling each node individually into *one* code executed atomically. From the user point-of-view, such a node becomes a combinatorial node whatever are the internal dependencies between its input and output. As a consequence, every feed-back loop must cross an explicit delay outside of the node. As is, this compilation method rejects causally correct programs (such as the *copy* example) which would have worked properly in a parallel execution or compiled with a white-box technique. This solution is the one taken in the compiler of SCADE and is combined to inlining *on demand* to accept all causally correct programs [3].

In this paper, we investigate an intermediate approach between the two that we call *grey-boxing*. It is based on the observation that some nodes are *compatible* in the sense that they can be executed together without restricting possible feedback loops between input and output. Then, the data-flow graph can be partitioned into a minimal number of sub-systems, each of them executed atomically. This idea originally appeared in a work by Raymond published in a french report in 1988 [13]. Left aside, the subject has been reconsidered recently by Lublinerman and Tripakis. Their first proposal [12] was based on dynamic testing to avoid the re-computation of shared values but, as they noticed in [11], this solution is costly in term of code size and execution time. In this recent work, Lublinerman, Szegedy and Tripakis come back to the original optimal static scheduling problem. They prove that the problem is intractable since it belongs to the family of optimization problems where the corresponding decision problem — does it exist a solution with at most c sub-systems? — is NP-complete as it encodes the *clique cover* in a graph. Then, the author derive an iterative algorithm looking for solutions for $c = 1, 2, ...$ where each step is encoded into a SAT problem.

Despite the apparent intractability of the problem, our experience is that real programs do not exhibit such a complexity. This call for a careful look at sources of combinatorial explosion and an efficient encoding which limits this possible explosion. Based on earlier work by Raymond on a study of input/output relations, this paper proposes a symbolic and efficient representation of the problem. We show that this encoding *simplifies* the problem in the sense that it rejects solutions but keeps *all* the optimal ones. This symbolic representation allows, in polynomial time, (1) to identify nodes for which several schedules are feasible and thus are possible sources of complexity; (2) to obtain solutions which in some cases are already optimal; (3) otherwise, to get a non trivial lower bound for c to start an iterative combinatorial search, once the symbolic representation is translated into a boolean formula. All this is proven using basic properties of partial orders.

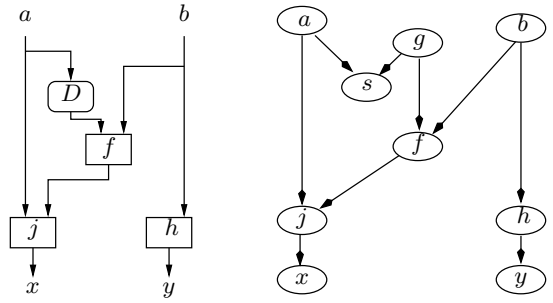


Figure 1: A data-flow network and the corresponding ordered set of actions

The paper is organized as follows. Section 2 presents the basics of the optimal scheduling problem and Section 3 develops the formalization. We start by giving another proof that optimal scheduling is intractable to help clarifying the sources of combinatorial explosion. In section 4, we present the symbolic representation based on input/output analysis, related properties and algorithms. Section 5 gives experimental results. We discuss related works in Section 6 and we conclude in section 7.

2. FROM SYNCHRONOUS DATA-FLOW TO RELATIONS

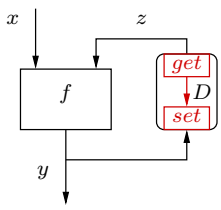
Consider the following typical synchronous data-flow program given below (and written in LUSTRE) syntax. The block `NODE` has two input `a`, `b` and two output `x` and `y`. Its body is made of two equations. Their dependencies are given on the left side of Figure 1.

```
node NODE(a, b: t) returns (x, y: t);
  let
    x = j(a, f(D(a), b));
    y = h(b);
  tel;
```

There is essentially two kind of operators in those networks, combinatorial and delay operators. Combinatorial ones (e.g., `j`, `f` and `h`) need their current input in order to produce their current output whereas a delay (e.g., `D`) is able to produce an output *before* it reads an input [5]. Delay nodes are used to program dynamical systems with feed-back loops.

Focusing only on the scheduling problem of data-flow net-

[5]The unitary delay is concretely written `pre` in LUSTRE and $1/z$ in (discrete) SIMULINK. From the dependencies point-of-view, an integrator $(1/s)$ in (continuous) SIMULINK acts similarly.



Possible sequential code:
$z = D.get()$
$y = f(x, z)$
$D.set(y)$

Figure 2: The get/set principle breaks feed-back loops

works with delays — and not memory representation and optimization — the simplest way to represent a delay D is to express it as two atomic imperative actions $D.get$ and $D.set$. $D.get$ returns the current value of the delay whereas $D.set$ reads the current input and stores it for remaining execution. More importantly, if the result of $D.get$ is necessary, it must be executed before $D.set$ is executed. Said differently, a delay *reverse* the data-flow dependency between its input and its output and thus breaks cycles (Figure 2). Using this lower-level representation, data-flow networks we consider are simply partially ordered sets of atomic actions (Figure 1). Precisely, a system $(A, I, O, \preceq)$ is made of a finite set of actions A, a subset of inputs $I \subseteq A$, a subset of output $O \subseteq A$ (not necessarily disjoint from I) and a partial order $\preceq$ to represent precedence relation between actions.

In such a data-flow network, it makes sense to feed-back an output with an input as soon as the output does not depend combinationally on this input. This is illustrated by the example of figure 3. If we allow such a feed-back loop, it is not possible to generate a single, monolithic code, for `NODE`: h cannot be computed in the same piece of code as j. However, it is clear that it is not necessary to externalize the code of each node in the graph, that is, to inline `NODE`. Some nodes can be gathered without preventing causally correct feed-back loops. The static scheduling problem is to generate the minimal number of classes of compatible nodes. For example, f can be computed:

- together with h, as soon as b is provided;
- together with (and before) j in order to provide the output x.

The same reasoning holds for the delay D:

- $D.get$ is clearly associated to the computation of f;
- $D.set$ requires the input a, and for this reason, is related to the action j.

Finally, this intuitive reasoning shows that the node can be abstracted into two atomic pieces of code:

- the class for h with input b and output y;
- the class for j with input a, output x, and which must always be computed after the class for `h`.

However, the frontier between these two classes is not strict: some internal computation can be performed on one side or the other, e.g., g and f. Figure 4 shows two particular optimal solutions in the number of classes. Once these blocks have been identified, we can compile each of them into sequential code executed atomically and export them together

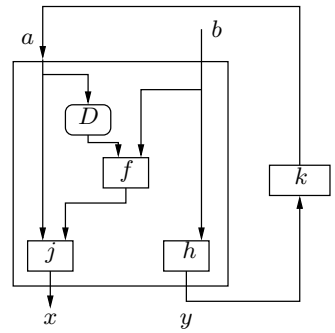


Figure 3: A correct feed-back use of a parallel component

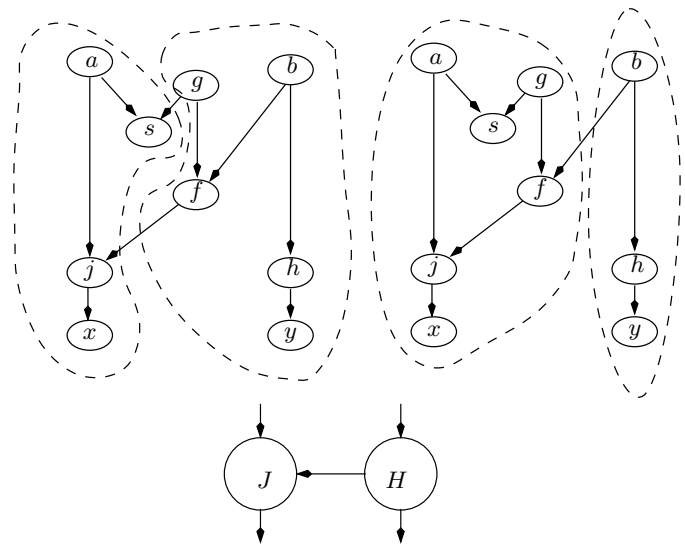


Figure 4: Two possible "grey-boxing" and optimal scheduling

with the relative dependence constraints. From the caller, `NODE` is now seen as a data-flow graph with two actions — the class of h and the class of j —. The call to `NODE` is replaced with a call to the two associated functions.

Note that the formulation with $(A, I, O, \preceq)$ is fully modular and enough to address the static scheduling problem. At each level of the hierarchy, we can focus on a set of actions, some of them being input and/or output and some being local. We now formalize the grey-boxing problem.

3. FORMALIZATION

3.1 Optimal static scheduling

Let A be a set of actions, partially ordered by $\preceq$, and two subsets of A, $I \subseteq A$ (inputs) $O \subseteq A$ (output).

Definition 1 (Compatibility). *Two actions $x, y \in A$ are said to be (static-scheduling) compatible and this is written $x \chi y$ when the following property hold:*

$$x \chi y \stackrel{def}{=} \forall i \in I, \forall o \in O, \quad ((i \preceq x \wedge y \preceq o) \Rightarrow (i \preceq o)) \wedge ((i \preceq y \wedge x \preceq o) \Rightarrow (i \preceq o))$$

Compatibility is the property which formalizes the fact

that two nodes (either internal, input or output) can be statically scheduled into a same piece of atomic code. If two actions are not compatible (for instance $i \preceq x$, $y \preceq o$ and $i \not\preceq o$), it is possible to feedback o to the input i without introducing a combinatorial loop. Thus, putting x and y into the same piece of code would make this feed-back loop impossible.

The goal is to find classes of pairwise compatible actions in A. Note that compatibility is not a solution, since it is not an equivalence. It is symmetric, reflexive, but not transitive in general: for example, in Figure 4, $f \chi j$ and $f \chi b$ hold but not $j \chi b$.

Moreover, any equivalence $\simeq$ included in χ is not a solution: in addition to be made of pairwise compatible actions, the atomic blocks must also be schedulable in such a way that the dependencies between (atomic) actions are satisfied. Moreover, they must not introduce any extra dependencies between input and output, in order to permit any feed-back which is allowed by the flat system.

In other terms, we are searching for an equivalence relation plus a (partial) order over the classes. This is strictly equivalent to search a *preorder* (reflexive, transitive, not symmetric relation) over the set of actions.

Definition 2 (Optimal Static Scheduling (OSS)). *A static schedule is a relation $\precsim \subseteq A \times A$ such that:*

- *(SS-0) $\precsim$ is a* preorder *(reflexive, transitive), and we note $\simeq$ the underlying equivalence relation ($x \simeq y \Leftrightarrow x \precsim y \wedge y \precsim x$),*
- *(SS-1) $x \preceq y \Rightarrow x \precsim y$ (it contains all the dependency constraints),*
- *(SS-2) $\forall i \in I, \forall o \in O, \quad i \precsim o \Leftrightarrow i \preceq o$ (the preorder strictly maps the dependency on input/output pairs).*

Moreover, a static schedule is said optimal (OSS) if it satisfies the following property:

- *(SS-3) $\simeq$ is maximal (i.e. it has a minimal number of classes).*

Property 1. *The main property following the definition SS-0 to SS-2 is that the equivalence $\simeq$ associated to a static scheduling $\precsim$ is a subset of the the compatibility relation:*

- *(SS-prop) $x \simeq y \Rightarrow x \ \chi \ y$*

Proof: Let $\precsim$ be a static scheduling, suppose that it exist some actions x, y, input i and output o such that $x \simeq y$, $i \preceq x$, $y \preceq o$ and $i \not\preceq o$, by SS-1, $i \precsim x$ and $y \precsim o$, then, since (in particular) $x \precsim y$, $i \precsim o$ and by SS-2 $i \preceq o$, which is absurd.

3.2 Theoretical complexity of OSS

An equivalent definition of the problem, presented in terms of graphs and clusters, has been proposed in [11]. Authors have shown that it is NP-hard, through a reduction of the *partition in k cliques* problem, also known as `clique cover` [5].

Even if it is not new, we briefly present here an alternative encoding which leads to the same result. This encoding is interesting since it helps to understand why the problem is theoretically so "hard", while it seems so simple on small, but however typical system such as the one of Figure 4.

We directly show that any *minimal clique cover* problem can be linearly encoded into a OSS instance. Firstly, we reformulate the problem in terms of relations: given a finite set L, and a symmetric relation $\leftrightarrow$ (i.e. a non oriented graph), find a maximal equivalence relation $\simeq$ included in $\leftrightarrow$ (i.e. with a minimal number of classes, called cliques in the original formulation).

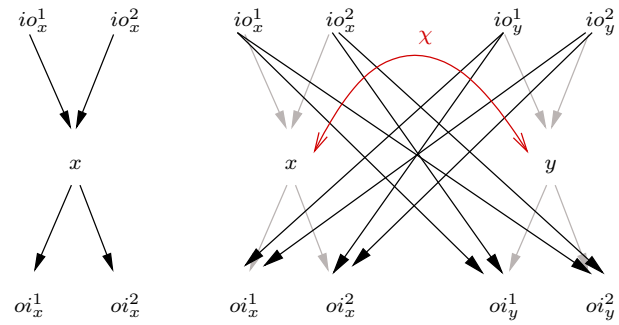


Figure 5: The X-encoding, isolated node (left), and related pair (right)

This problem has indeed many similarities with the OSS one since OSS mainly consists in finding a maximal equivalence included in a symmetric relation (the compatibility). However, it is not trivial that compatibility relations can be as general as symmetric relations.

The X-encoding.

In order to prove it, we propose an encoding of MCC into OSS (called X-encoding). Let $G = (L, \leftrightarrow)$ be the data of a MCC problem. We build a OSS instance with an extra set of nodes X: $(A = L \uplus X, I = O = X, \preceq)$ (i.e. each extra variable is both an input and an output), in the following way:

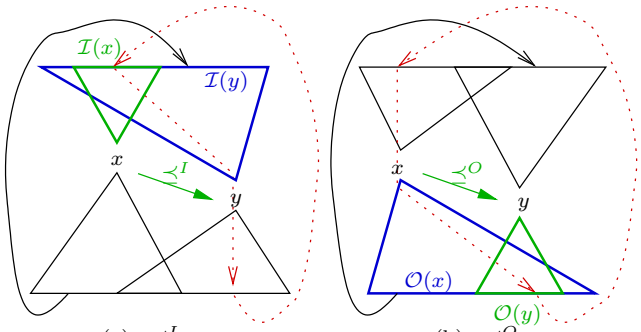
1. For each $x \in L$, we introduce 4 extra variables io_x^1, io_x^2, oi_x^1 and oi_x^2 that are *both* input and output. Each local variable x and its extra input/output variables are related by the dependency relation in a "X" (cross) shape, as presented on the left side of Figure 5.

2. For each pair x, y such that $x \leftrightarrow y$, we add 8 dependencies by connecting each input of x to each output of y and vice versa, as shown on the right side of Figure 5.

The X-encoding ensures that each additional input-output is incompatible with any other variable, including the corresponding local x. Moreover, the dependencies added for each pair $x \leftrightarrow y$ enforce the compatibility of these variables. Finally, we have $x \chi y \Leftrightarrow x \leftrightarrow y$: it establishes that a compatibility relation can be an arbitrary symmetric relation. Moreover, whatever be an input io_x of a local node x and an output oi_y of a local node y, we have, by construction, $x \chi y \Rightarrow io_x \preceq oi_y$: it establishes that, whatever be a preorder $\precsim$ included in $\preceq \cup \chi$, then $io \precsim oi \Leftrightarrow io \preceq oi$ (or, more intuitively, whatever be the way local variables are gathered into "cliques", it won't add any extra dependency between input and output).

It follows that: whatever be $\simeq$ an (optimal) solution of the clique cover problem, then $\simeq \cup \preceq$ is an optimal solution of the corresponding X-encoded problem, and, whatever be $\precsim$ an (optimal) solution of the X-encoded problem, then the associated equivalence $\simeq$ is an optimal solution of the clique cover problem.

A main interest of this encoding is that is outlines the fact that, even if OSS is proven to be at least as complex as



(a) $x \preceq^I y$ (b) $x \preceq^O y$

In both case: x may be "fed-back" to y, but y cannot

Figure 6: Pre-orders $x \preceq^I y$ and $x \preceq^O y$ give extra information on feasible scheduling.

MCC, the instances of OSS that are supposed to be "hard" are far to what is intuitively a data-flow program: in order to feet the general case, we have to consider programs where the number of input/output is very large compared to the number of internal nodes, and, moreover, the dependencies between input and output are somehow "chaotic".

Our purpose is then to establish that the complexity of OSS is strongly related to the number of input/output, and the structure of their dependencies.

4. INPUT/OUTPUT ANALYSIS

This section shows that the OSS problem can be reformulated, and in some sense simplified, by analyzing the relations between input and output.

4.1 Input/output saturation

Let $(A, I, O, \preceq)$ be a data-flow network.

Comparing the sets of inputs (resp. outputs) of two nodes may give informations on how they can be statically scheduled without forbidding valid feed-backs. As shown in Figure 6:

- (a) if the inputs of x ($\mathcal{I}(x)$) are included in the inputs of y ($\mathcal{I}(y)$), then any feed-back from an output of y to some input of x creates a combinational loop. As a consequence, it is never the case that y *should be computed* before x, or, in other terms, x can always be computed before y.

- (b) the same reasoning holds with the (reverse) inclusion of outputs: if $\mathcal{O}(x) \supseteq \mathcal{O}(y)$ then x can always be computed before y.

These relations $\preceq^I$and $\preceq^O$(that are trivially preorders), are called respectively the *input* (resp. *output*) saturations of the relation $\preceq$. They are solutions of SS:

- (SS-1) they both include $\preceq$, since $x \preceq y$ implies $\mathcal{I}(x) \subseteq \mathcal{I}(y)$ and $\mathcal{O}(x) \supseteq \mathcal{O}(y)$,

- (SS-2) they do not introduce any extra input/output dependency, since $\mathcal{O}(i) \supseteq \mathcal{O}(o)$, and, in particular, $o \in \mathcal{O}(o)$, we have $i \preceq o$ (similarly for the inclusion of input).

It follows that, from the basic dependency relation (which is a non optimal solution of SS), one can derive another solution which is still non optimal, but better than the original.

In fact, the same reasoning holds when starting from any preorder which is a solution of SS.

Definition 3 (Input and output saturation). *Given a solution $\precsim$ of the SS problem, we define its input (resp. output) function, saturation preorder and equivalence:*

$$\begin{array}{rcl}
\mathcal{I}^{\precsim}(x) & = & \{i \in I\,, i \precsim x\} \\
x \precsim^I y & \Leftrightarrow & \mathcal{I}^{\precsim}(x) \subseteq \mathcal{I}^{\precsim}(y) \\
x \simeq^I y & \Leftrightarrow & \mathcal{I}^{\precsim}(x) = \mathcal{I}^{\precsim}(y) \\
\mathcal{O}^{\precsim}(x) & = & \{o \in O\,, x \precsim o\} \\
x \precsim^O y & \Leftrightarrow & \mathcal{O}^{\precsim}(x) \supseteq \mathcal{O}^{\precsim}(y) \\
x \simeq^O y & \Leftrightarrow & \mathcal{O}^{\precsim}(x) = \mathcal{O}^{\precsim}(y)
\end{array}$$

We list the main properties of the saturation preorders, and detail the proof only for the input-saturation $\precsim^I$ (the case of $\precsim^O$ is dual):

Property 2. *$\precsim^I$ and $\precsim^O$ are solution of SS.*

Proof for $\precsim^I$:

- *(SS-1) since $\precsim$ is itself a solution of SS, $x \preceq y \Rightarrow x \precsim y$, and $x \precsim y \Rightarrow \mathcal{I}^{\precsim}(x) \subseteq \mathcal{I}^{\precsim}(y)$*

- *(SS-2) if $\mathcal{I}^{\precsim}(i) \subseteq \mathcal{I}^{\precsim}(o)$ then, since in particular $i \in \mathcal{I}^{\precsim}(i)$, we have $i \precsim o$, which implies $i \preceq o$ since $\precsim$ satisfies SS-3*

Property 3. *for any SS solution, input-saturation equals compatibility on output, and output-saturation equals compatibility on input:*

$$\begin{array}{rcl}
\forall o, o' \in O\ o \precsim^I o' & \Leftrightarrow & o \simeq^I o' \Leftrightarrow o \chi\, o' \\
\forall i, i' \in I\ i \precsim^O i' & \Leftrightarrow & i \simeq^O i' \Leftrightarrow i \chi\, i'
\end{array}$$

Proof for $\precsim^I$, for all output o, o':

- *$o \simeq^I o' \Rightarrow o \chi\, o'$, because of SS-prop,*

- *to prove the reverse, since χ is symmetric, we simply prove that $o \chi\, o' \Rightarrow o \precsim^I o'$.*

  If $o \not\precsim^I o'$, then $\exists i \in I$ such that $i \precsim o$ and $i \not\precsim o'$, from SS-2, we have also $i \preceq o$ and $i \not\preceq o'$, and then o and o' are not compatible.

Property 4. *In any* optimal solution *of SS, two inputs (resp. outputs) that are compatible are necessarily in the same class.*

Proof for output pairs. Suppose that $\precsim$ is an optimal solution of SS, where two outputs are not equivalent ($o \not\simeq o'$) then compute its input-saturation $\simeq^I$. From the properties of $\simeq^I$, $o \simeq^I o'$, and then $\simeq^I$ and $\simeq$ are different. Moreover $\simeq^I$ is greater or equal ($\simeq^I \supseteq \simeq$) than $\simeq$. Finally, $\simeq^I$ has strictly less classes than the supposed optimal solution, which is absurd. The proof is similar for input pairs.

Indeed, saturation "works" only once: let $\precsim$ be a SS solution and $\precsim^I$ its input-saturation, then $i \precsim^I x \Leftrightarrow i \precsim x$: input according to $\precsim^I$ are exactly output according to $\precsim$. Similarly for output-saturation.

However, performing input-saturation then output-saturation (or the reverse) may give some extra information.

Definition 4 (Input/output preorder). *Let $\precsim$ be a solution of SS, we define its input/output saturation $\precsim^{IO}$, and equivalence $\simeq^{IO}$, as the input-saturation of its output-saturation:*

$$\begin{array}{rcl} \mathcal{I}^{\precsim}_{\mathcal{O}}(x) & = & \{i \in I \,, i \precsim^{O} x\} \\ x \precsim^{IO} y & \Leftrightarrow & \mathcal{I}^{\precsim}_{\mathcal{O}}(x) \subseteq \mathcal{I}^{\precsim}_{\mathcal{O}}(y) \\ x \simeq^{IO} y & \Leftrightarrow & \mathcal{I}^{\precsim}_{\mathcal{O}}(x) = \mathcal{I}^{\precsim}_{\mathcal{O}}(y) \end{array}$$

This preorder inherits from the properties of both saturations: (1) it is a solution of SS, (2) it meets compatibility on inputs ($i \chi i' \Leftrightarrow i \simeq^{IO} i'$), (3) it meets compatibility on outputs ($o \chi o' \Leftrightarrow o \simeq^{IO} o'$).

Moreover, it has a new property on input/output pairs:

Property 5. *$\simeq^{IO}$ meets compatibility on $I \times O$:*

$$\forall i \in I, \forall o \in O, i \chi o \;\Leftrightarrow i \simeq^{IO} o$$

Proof:

- *$i \simeq^{IO} o$ implies $i \chi o$ from SS-prop.*
- *$i \chi o \Rightarrow i \preceq o$ by definition, $i \preceq o \Leftrightarrow i \precsim^{IO} o$ from SS-2.*
- *$i \chi o \Rightarrow o \precsim^{IO} i$, because, if $o \not\precsim^{IO} i$ it exists i' such that (a) $i' \precsim^{O} o$ and (b) $i' \not\precsim^{O} i$; from (a) and SS-2 $i' \precsim o$ and from (b) $\exists o'$ $i \precsim o' \;\wedge\; i' \not\precsim o'$, thus, from SS-2, i and o are not compatible.*

The consequence of these properties is that, in any optimal solution of the SS problem, any pair of compatible nodes in $I \cup O$ are necessarily in the same equivalence class.

Moreover these equivalence classes on $I \cup O$ can be computed from any known solution of SS, in particular from the trivial "worst" solution $\preceq$. We note $\mathcal{I}_{\mathcal{O}}$ and $\preceq^{IO}$ the input-output function and saturation of $\preceq$.

In other term, there is no choice for "clustering" inputs and outputs in an optimal solution, and, moreover, computing $\mathcal{I}_{\mathcal{O}}$ gives a lower-bound to the number of classes.

However, there remain several problems to address: where to put local nodes in an optimal solution? when additional classes are necessary? and indeed, how many additional classes are necessary?

Our goal in the sequel is to precise and solve these problems, while trying to identify "simple" cases where the answers are easy, and the hard ones, that fall into the NP-hard theoretical complexity.

4.2 Static Scheduling as a mapping in the input power-set

In this section, we show that solving (O)SS is equivalent to finding a (minimal) mapping of the actions into the set of input parts.

Any definition, reasoning and result can be trivially adapted to the dual case of mapping actions to subsets of outputs.

Consider a mapping $\mathcal{K} : A \mapsto 2^I$, which satisfies the following properties:

- (KI-1) $\forall x \in I \cup O, \;\; \mathcal{K}(x) = \mathcal{I}_{\mathcal{O}}(x)$
- (KI-2) $\forall x, y, \;\; x \preceq y \;\Rightarrow\; \mathcal{K}(x) \subseteq \mathcal{K}(y)$

The problem of finding a mapping satisfying these properties is called the *Keys as inputs subsets* encoding (or KI-enc). Obviously, the dual problem of *Keys as outputs* is equivalent. We show that this problem solves the SS one, in the sense that a solution of KI-enc directly leads to a solution of SS.

KI-enc solves SS: Indeed, the mapping $\mathcal{I}_{\mathcal{O}}$ is a trivial solution of KI-enc, and it gives, through keys inclusion, a solution of SS which is simply $\preceq^{IO}$.

But the result is much general: let $\mathcal{K}$ be any solution of KI, then the preorder $x \precsim y \Leftrightarrow \mathcal{K}(x) \subseteq \mathcal{K}(y)$ is a solution of SS:

- SS-1 follows from KI-2,
- SS-2 follows from KI-1 and the properties of $\mathcal{I}_{\mathcal{O}}$.

SS solves (and is improved by) KI-enc: suppose that $\precsim$ is a solution of SS, apply output and then input saturation, the corresponding $\mathcal{I}^{\precsim}_{\mathcal{O}}$ is a solution of KI-enc, and, moreover the corresponding preorder is equal or better (i.e. included) than $\precsim$.

These properties are interesting for the search of optimal solutions, that is, preorders with a minimum number of classes. They imply that (1) any solution of KI-enc gives a solution of SS, (2) from any solution of SS, one can build a *better or identical* solution of KI-enc. In other terms, the KI-enc formulation has strictly less solutions than SS, but it does not miss *any optimal solution.*

4.3 Computing the KI-enc system

We reformulate here the KI-enc problem into a more computational problem. We note $\rightarrow$ the direct strict dependency, which is indeed the natural way to represent concisely a partial order (i.e. minimal dependency graph):

$$x \rightarrow y \;\Leftrightarrow\; x \neq y \wedge x \preceq y \wedge \forall z \; z \preceq y \Rightarrow (z \preceq x \;\vee\; x \not\preceq z)$$

Then, we build a system of (in)equations with a variable K_x for each $x \in A$:

- $K_x = \mathcal{I}_{\mathcal{O}}(x)$ for $x \in I \cup O$
- $\bigcup\limits_{y \rightarrow x} K_y \;\subseteq\; K_x \;\subseteq\; \bigcap\limits_{x \rightarrow z} K_z$ otherwise

Where $\bigcup$ and $\bigcap$ are the natural n-ary extensions of union and intersection, that is, they map 0-argument to their respective neutral element: $\bigcup\limits_{\emptyset} = \emptyset$, and $\bigcap\limits_{\emptyset} = I$.

Complexity analysis.

The complexity of this building is strictly due to the computation of $\mathcal{I}_{\mathcal{O}}$. More precisely, let n, n_i, n_o be the number of nodes, inputs, outputs:

- computing the output function $\mathcal{O}$ can be done by assigning to each node a set variable O_x, and then by visiting once, in a bottom-up topological order, the p arcs of the dependency graph. For each arc $x \rightarrow y$, a set union must be made ($O_x := O_x \cup O_y$) whose cost is theoretically in $n_o . \log n_o$ (negligible in practice for small values of n_o, when encoding sets with bit-vectors). If the dependency graph is almost minimal, we can state that p is of the same order than n, and then the global cost is of the order $n.n_o . \log n_o$.
- given $\mathcal{O}$, the $\mathcal{I}_{\mathcal{O}}$ of each node x can be computed by comparing $\mathcal{O}(x)$ to $\mathcal{O}(i)$ (cost in $n_o . \log n_o$), if it is included, i is added to the known $\mathcal{I}_{\mathcal{O}}$ of x (cost in $\log n_i$). The cost is globally of the order $n.n_i . \log n_i . n_o . \log n_o$.

Finally, the system can be built in (roughly) $n.m^2.(\log m)^2$, where n is the number of nodes, m is the maximum between the number of inputs and outputs.

If we isolate the costs of set operations ($B(m)$ for binary ones, and $A(m)$ for insertion), we obtain the expression $n.m.B(m).A(m)$ which gives a more interesting information of the cost in practice.

Iterating SAT searching.

The system is trivially a SAT problem, since a subset of I can be represented by a vector of n_i Booleans:

- each variable K_x can be encoded by a vector of Boolean variables $[K_x^1, \cdots, K_x^{n_i}]$
- each constant $\mathcal{I}_\mathcal{O}(x)$ can be replace by the corresponding vector of n_i bits,
- $\bigcup$ is the bit-to-bit logical or, $\bigcap$ is the bit-to-bit logical and.

As we have shown in section 4.1, the system outlines the fact that some classes are necessary: the ones corresponding to the $\mathcal{I}_\mathcal{O}$ of inputs and outputs. We call them the mandatory classes $M_1, ...M_c$. An optimal solution can be obtained by iterating SAT-solver calls:

- search a solution with $c+0$ classes, by adding, for each local variable x, the constraint $\bigvee_{j=1}^{j=c}(K_x = M_j)$, and calling a SAT solver,
- if it fails, introduce a new n_i *vector of free variables* S_1, and add this variable vector as a new possible value for each local x:
$$\bigvee_{j=1}^{j=c}(K_x = M_j) \vee (K_x = S_1)$$
- if it fails, introduce another extra variable S_2, and so on.

However, before using this costly iterative search, it may be interesting to study more precisely the system, and identify the sources of complexity.

4.4 Simplifying the KI-enc system

Lower and higher bounds.

For each variable, we define the lower and the upper solution:

- $k_x^\perp = k_x^\top = \mathcal{I}_\mathcal{O}(x)$ for $x \in I \cup O$
- $k_x^\perp = \bigcup_{y \longrightarrow x} k_y^\perp$ and $k_x^\top = \bigcap_{x \longrightarrow z} k_z^\top$ otherwise.

It is easily shown that $k_x^\top$ is in fact $\mathcal{I}_\mathcal{O}(x)$: setting all variables to their upper bound simply correspond to the already known solution $\preceq^{I_O}$. It corresponds to the heuristic "schedule as late as possible"; we note this solution $\mathcal{K}^\top$. Dually, choosing $K_x = k_x^\perp$ for each node is also trivially a solution, which intuitively means "schedule as soon as possible" (noted $\mathcal{K}^\perp$). Indeed these solutions are *a priori* incomparable, and not optimal in general. However, if it appears that all $k_x^\top$ (resp. $k_x^\perp$) are keys of mandatory classes (i.e. keys of some input or output), we can already conclude that $\mathcal{K}^\top$ (resp, $\mathcal{K}^\perp$) is optimal.

In some cases, the bounds may precise the optimal solution. Whenever $k_x^\perp = k_x^\top$, the inequation becomes an equation, and the class becomes mandatory, even if it not the class of any input or output.

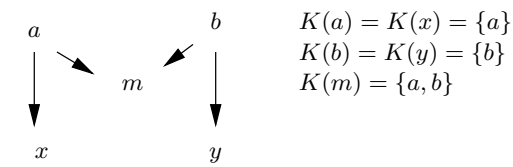


Figure 7: A simple "M" shape, where $I = \{a, b\}$ and $O = \{x, y\}$.

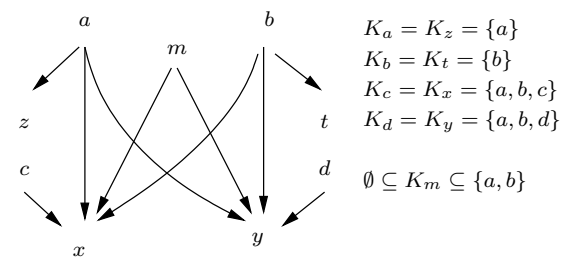


Figure 8: A "M/W" shape, where $I = \{a, b, c, d\}$ and $O = \{x, y, z, t\}$.

This case corresponds to the one where it is easy to decide if some purely internal class is necessary. The simplest example of such a case is shown in Figure 7: the local m must be computed alone, in a class that should be scheduled after the ones of a and b.

Bounds reduction.

The bounds can be introduced into the inequalities, whose general form is then:

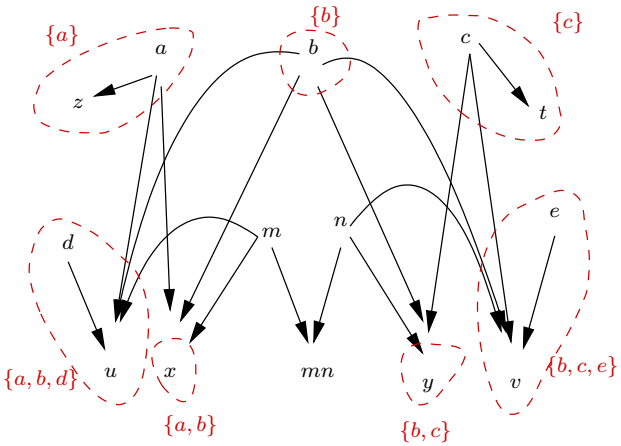
$$k_x^\perp \cup \bigcup_{y \longrightarrow x} K_y \subseteq K_x \subseteq k_x^\top \cap \bigcap_{x \longrightarrow z} K_z$$

For any $y \rightarrow x$ such that $k_y^\top \subseteq k_x^\perp$, the constraint $K_y \subseteq K_x$ is redundant and can be removed; similarly for any $x \rightarrow z$ such that $k_x^\top \subseteq k_z^\perp$. This simplification is called the (redundant) bounds reduction.

With this reduction, it may appear than all variables are removed in a inequation, and thus, that some variables becomes bounded by constants:
$k_x^\perp \subseteq K_x \subseteq k_x^\top$
Whenever $k_x^\perp \neq k_x^\top$ for some x, there are several choices for the "position" of x within the static schedule, that may lead or not to an optimal solution. Figure 8 shows a small example with a "M/W" shape where the position of the local variable m is bounded by two constants. In this case, no extra class is necessary if the intersection of all the intervals contains a mandatory class. In this example, the answer is trivial since we have only one interval. It includes both the mandatory classes $\{a\}$ and $\{b\}$ and one can choose either $K_m = \{a\}$ or $K_m = \{b\}$ to obtain an optimal solution.

As soon as there are more than 3 such variables, we fall in a case that is equivalent to the one of the X-encoded problem: n independent internal variables have to be organized in a minimum number of classes, according to their bounds.

The problem is even more complex when the bounds are not constant: the position of some local variable may depend



$$\emptyset \subseteq K_m \subseteq \{a,b\} \cap K_{mn}$$
$$\emptyset \subseteq K_n \subseteq \{b,c\} \cap K_{mn}$$
$$K_m \cup K_n \subseteq K_{mn} \subseteq \{a,b,c,d,e\}$$

Figure 9: A generalized "M/W" shape, where $I = \{a,b,c,d,e\}$ and $O = \{x,y,z,t,u,v\}$.

on the position of another and so on. Figure 9 illustrates this case by generalizing the "M/W" shape. Several optimal choices can be made, that do not require extra classes, for instance $K_m = K_n = K_{mn} = \{b\}$, but also $K_m = \{a\}, K_n = \{b\}, K_{mn} = \{a,b\}$ and more. But these optimal solutions cannot be obtained by considering variables one by one. For instance, $K_m = \{a\}$ is a (locally) good choice, such as $K_n = \{c\}$; but once these two choices have been made, there is no other solution for mn than $K_{mn} = \{a,c\}$ which obliges to introduce a extra class. Intuitively, when several variables are related, they must be considered "all together", making the problem computationally hard.

As a conclusion, the proposed encoding may, in some cases, directly gives solutions that are proven optimal, and otherwise it outlines the parts of the systems that are *real* sources of complexity: the internal variables that are bounded by non trivial intervals, possibly parameterized by other variables.

5. EXPERIMENTATION

We have developed a prototype in order to experiment the ideas presented in the paper. The goal is not to have an efficient tool that can be integrated in a compiler, but just to see if modular compilation is actually complex on real industrial programs [6].

First, the dependency informations from a SCADE (or LUSTRE) program are extracted, then the prototype builds the KI-system and checks whether:

- it is only made of equations, in which case the system is already solved; we call it the *trivial* case: i.e. the position of each variable is mandatory;
- it contains inequalities, but either $\mathcal{K}^\top$ or $\mathcal{K}^\perp$ is an optimal solution; in this case the system is said *solved*. It corresponds to the case where a simple heuristic leads to an optimal solution (always compute as soon as possible, or always compute as late as possible).
- otherwise, the system is considered as complex; the set of mandatory classes M (and thus its cardinal m), and the variables that are really problematic V are identified. The tools generates a generic Boolean formula (as presented in 4.3) parameterized by the number of extra classes c. This formula can be expanded for $c = 0, 1 \cdots$, and then given to a Boolean solver.

[6]Even not optimized, the prototype requires respectively 2, 0.5 and 10 seconds on a laptop (centrino 1.2 Ghz) to treat the whole 3 benchmarks presented here.

SCADE libs	trivials	solved	others
n. of programs (155)	41	114	0
n. of classes	1	1 to 2	-
n. of in/out	2 to 5	2 to 9	-
n. of nodes	2 to 18	8 to 64	-
av. size	10	24	-

Bench 2	trivials	solved	others
n. of programs (27)	8	19	0
n. of classes	1	1 to 4	-
n. of in/out	2 to 10	2 to 19	-
n. of nodes	2 to 29	9 to 48	-
av. size	14	20	-

Bench 3	trivials	solved	others
n. of programs (125)	39	85	1
n. of classes	1 to 3	1 to 4	3 or 4
n. of in/out	2 to 14	2 to 26	4+2
n. of nodes	7 to 350	12 to 600	25
av. size	50	80	-

Results show the ranges of the number of classes (i.e. the size of the optimal solution); the programs in a particular benchmark are of various sizes, we give the ranges of the number of inputs/outputs and of internal nodes, and also the average size (in internal nodes).

Figure 10: Experimental results: almost 100% of the programs are polynomially solved.

Benchmark 1 (Scade Lib)..

The problem of modular compilation is clearly interesting for programs that are supposed to be reusable. This is why we have chosen, as a first benchmark, the libraries of the SCADE tool [7]. The benchmark is made of 155 block diagrams (we simply call them programs in the sequel), which are indeed relatively small. Not surprisingly, none of them was identified as "complex" (see Figure 10). Moreover, 88% of the programs are combinational: each output depends on all inputs (e.g. math libs); as a consequence they trivially require a single computation block. The others operators (19 of 155) are basic temporal operators (e.g. integrators) that require 2 blocks.

Since it seems very unlikely to find complex reusable programs, we have tried to apply the method on bigger industrial examples.

[7]For practical reasons, we have used the libraries of the version 4.2

Benchmark 2..

This benchmark is extracted from a (medium) industrial application coming form Airbus. We have extracted 27 components and sub-components of a medium sized application (about 600 atomic nodes when flattened). Indeed, performing modular compilation is not really relevant in this case since the components are not intended to be reused. But the goal here is just to try our method on programs that are relatively bigger than those that can be found in libraries. Finally, the conclusion is the same as in the first benchmark: all the programs are solved by a simple input/output analysis. One particular program is interesting: it has 11 inputs and 4 outputs for a total of 30 nodes, and its optimal static scheduling has 4 classes, which is for the time being a (very relative) record of complexity.

Benchmark 3..

The last benchmark is also extracted from an Airbus application, but it is bigger than the previous one: 125 components are extracted from an application whose "flat" size is about 8000 nodes. Once again, the components are more tasks and sub-tasks than reusable operators, and they are not obviously candidates for separate compilation.

The conclusion is the same than previously, except that we have found a program that is not solved (cf. column *others* in the last table of Figure 10). This program has 24 nodes, 4 inputs and 2 outputs, the minimal number of (mandatory) classes is 3, and both $\mathcal{K}^{\top}$ and $\mathcal{K}^{\perp}$ give 4 classes. The corresponding equations system is simplified, and the tool isolates 10 internal variables whose position is not clear. Figure 11 shows the bounds and the remaining inter-dependencies of these variables.

```
{}  <= K(g1)  <=  {d, a}
{a} <= K(v10) <=  {d, a}              K(v3)  <= K(s0)
{a} <= K(s1)  <=  {d, c, b, a}        K(v3)  <= K(v4)
{}  <= K(g0)  <=  {a}                 K(v9)  <= K(v11)
{}  <= K(v2)  <=  {a}                 K(v10) <= K(s1)
{}  <= K(v9)  <=  {d, a}              K(v10) <= K(v11)
{a} <= K(v11) <=  {d, a}              K(g0)  <= K(v2)
{a} <= K(v3)  <=  {c, b, a}           K(g1)  <= K(s1)
{a} <= K(s0)  <=  {d, c, b, a}        K(g1)  <= K(v9)
{a} <= K(v4)  <=  {c, b, a}
```

Figure 11: System of inequations of a (possibly) complex program, as it is echoed by the prototype: the constants bounds (left) and the variables constraints (right).

It appears that in fact, it is also a simple problem, but it requires a heuristic that is not (yet) implemented in the prototype:

- the intersection of all the intervals is not empty ($\{a\}$),
- moreover the class $\{a\}$ is one of the 3 mandatory classes,
- thus, setting all the variable to $\{a\}$ satisfies all the constraints without introducing any extra class.

Conclusion.

Is it indeed possible to build arbitrarily complex data-flow network: just build a huge network by multiplying the M/W shape of Figure 9. However, our intuition is that "human-designed" data-flow networks are necessarily "simple", in particular for all that concerns its interface.

If it is admitted that modular compilation is useful (problem which is briefly discussed in the next section), the proposed approach consists in using an *inexpensive* algorithm that:

- in most cases, gives an optimal solution,
- in some cases, identify the problem as potentially "hard" and calls a third party non-polynomial tool for solving it.

6. DISCUSSION AND RELATED WORKS

Various approaches to the code generation problem of synchronous data-flow diagrams have been experimented in compilers.

Modular code generation, as followed by the SCADE compiler, essentially translates every stream function into a single function [3]. This way, modules can be compiled separately, calling conventions (to build libraries or to link synchronous code with others) are almost trivial and simplifies traceability between the source and target code. It appears that extra constraints imposed on feed-back loops are well accepted by users [8]. Combined to partial inlining, all remaining causally correct programs can be accepted. Note that in tools such as SCADE (but it also applies to SIMULINK), many functions are local as they are only used to structure the code: they are not exported to build libraries and only instantiated once. Any good compiler always inline these functions thus reducing again the set of functions which need a decomposition into sub-systems. A classical variation of the unique transition function is to split it in two, one to produce the output and one to modify the state [9]. In the case of a node where output do not depend on input (as for the delay node), it allows to compile modularly an extra set of programs but this solution is incomplete (e.g., it fails to compile the introductory example).

This approach to code generation was almost discarded in the LUSTRE compiler as it may reject causally correct programs. Programs are compiled after a full inlining of functions and this allows to generate efficient automata [8].

The question of decomposing a data-flow graph into classes has been also studied by Benveniste & al. for the compilation of SIGNAL and was observed to be similar to the problem of code distribution [2]. It is formulated for a more expressive model of conditional scheduling constraints: a relation $x \xrightarrow{c} y$ states that that "when c is true, y must be executed after x". A graph can be scheduled when all cycles $x_1 \xrightarrow{c_1} x_2 \xrightarrow{c_2} ...x_1$ are such that $\neg(c_1 \wedge ... \wedge c_n)$. We thus have considered the case where the $c_i = 1$. Nonetheless, the question of producing an optimal solution in number of classes does not seem to have been addressed.

We have already pointed the main differences with the work of Lublinerman, Szedezy and Tripakis [11]. The optimal solution is obtained iteratively $c = 1, 2, ...$ with a general solution based on a SAT encoding of a decomposition into c sub-systems. Nonetheless, real programs have simple dependency relations and can be treated with a dedicated algorithm with mastered worst-case complexity. In

[8]This is also due to the nature of applications written in SCADE where closed loops are programmed to be robust to the insertion of a delay.

[9]This approaches is convenient when the code is linked to a numerical variable step solver as it simplifies the rollback mechanism.

most applications we have found, the optimal is obtained in polynomial time using the direct algorithm based on the input/output relations. Even in the case where an iterative search would have to be done, it allows to start with a non trivial lower bound for c.

Finally, it is not clear that obtaining an optimal solution is central. In all the examples we have treated $\mathcal{K}^{\perp}$ and/or $\mathcal{K}^{\top}$ allows to obtain either the optimal or a solution close to the optimal [10].

7. CONCLUSION

This paper addressed the static scheduling problem of a synchronous data-flow network to be used in a compilation into sequential code. Precisely, how to decompose a data-flow network into a minimal number of sub-systems executed atomically without restricting possible feed-back loops between input and output? Though this optimization problem is intractable in the general case and be tackled with general combinatorial methods, our experience is that real programs do not reveal such a complexity. Based on the notion of input/output properties, we build, in polynomial time, a symbolic representation (keys as inputs subsets, or KI-enc) of the problem. This representation *simplifies* the problem in the sense that it has strictly less solutions but it contains *all* optimal ones.

This representation directly gives: (1) a non trivial lower-bound for the number of classes in any solution, (2) two particular solutions $\mathcal{K}^{\perp}$ and $\mathcal{K}^{\top}$. It can then easily checked whether $\mathcal{K}^{\perp}$ and/or $\mathcal{K}^{\top}$ are optimal by just comparing their number of classes with the lower bound. If it is not the case, an optimal solution is obtained by a iterative sat-solving search (is there exist a solution with min classes? min+1 classes etc.). In most programs we have encountred (more precisely all but one), $\mathcal{K}^{\perp}$ appeared to be optimal, and thus, the iterative sat-solving search were not necessary.

Acknowledgements.

We thank Léonard Gérard and anonymous reviewers for their carefull reading and comments.

8. REFERENCES

[1] A. Benveniste, P. LeGuernic, and Ch. Jacquemot. Synchronous programming with events and relations: the SIGNAL language and its semantics. *Science of Computer Programming*, 16:103–149, 1991.

[2] Albert Benveniste, Paul Le Guernic, and Pascal Aubry. Compositionality in dataflow synchronous languages: Specification & code generation. Technical Report R3310, INRIA, November 1997.

[3] Darek Biernacki, Jean-Louis Colaco, Grégoire Hamon, and Marc Pouzet. Clock-directed Modular Code Generation of Synchronous Data-flow Languages. In *ACM International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES)*, Tucson, Arizona, June 2008. Extended version of [4].

[4] Darek Biernacki, Jean-Louis Colaco, and Marc Pouzet. Clock-directed Modular Compilation from Synchronous Block-diagrams. In *Workshop on Automatic Program Generation for Embedded Systems (APGES)*, Salzburg, Austria, october 2007. Embedded System Week.

[5] M. R. Garey and D. S. Johnson. *Computers and Intractability - A guide to the Theory of NP-completeness*. Freeman, New-York, 1979.

[6] Georges Gonthier. *Sémantiques et modèles d'exécution des langages réactifs synchrones*. PhD thesis, Université Paris VI, Paris, 1988.

[7] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud. The synchronous dataflow programming language LUSTRE. *Proceedings of the IEEE*, 79(9):1305–1320, September 1991.

[8] N. Halbwachs, P. Raymond, and C. Ratel. Generating efficient code from data-flow programs. In *Third International Symposium on Programming Language Implementation and Logic Programming*, Passau (Germany), August 1991.

[9] Gilles Kahn. The semantics of a simple language for parallel programming. In *IFIP 74 Congress*. North Holland, Amsterdam, 1974.

[10] E. A. Lee and D. G. Messerschmitt. Static scheduling of synchronous data flow programs for digital signal processing. *IEEE Trans. on Computers*, 36(2), 1987.

[11] R. Lublinerman, C. Szegedy, and S. Tripakis. Modular Code Generation from Synchronous Block Diagrams — Modularity vs. Code Size. In *ACM Principles of Programming Languages (POPL)*, 2009.

[12] R. Lublinerman and S. Tripakis. Modularity vs. reusability: Code generation from synchronous block diagrams. In *Design, Automation and Test in Europe (DATE'08)*, 2008.

[13] Pascal Raymond. Compilation séparée de programmes lustre. Technical report, Projet SPECTRE, IMAG, juillet 1988. `url="http://www-verimag.imag.fr/SYNCHRONE/publis/spectre-L5.pdf"`.

[10] There is a similarity here with graph coloring for register allocation. Through it is an NP-complete problem, compiler mostly use sub-optimal greedy algorithms which give reasonably good results in practice.